# LSTAT2150 Project - Template A

Nathan Devaux

2025-12-23

## 1 Exploratory analysis and kernel density estimation

We consider $n = 250$ observations $(X_i, Y_i)$, $i = 1, \ldots, n$, drawn from a large population of households, where $X_i$ represents the day of the year (measured as a continuous random variables between 0 and 365) and $Y_i$ represents the total heating demand (in kWh/day) for that household on day $X_i$.

```r
### Load your data here using:
name <- "40852100-template-A.RData"
load(name)
head(data)
```

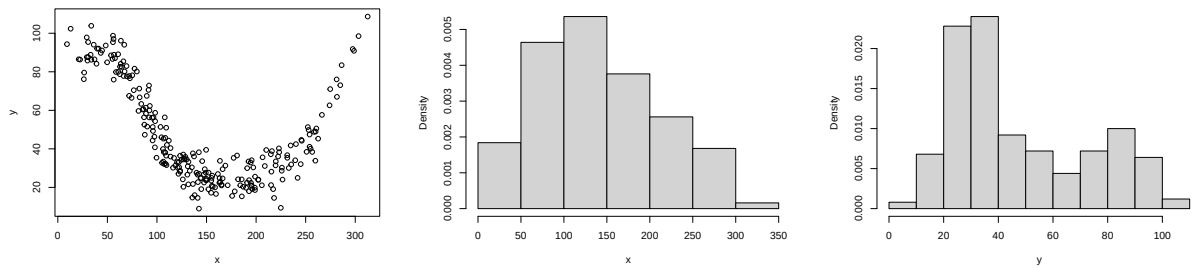
```
##           x        y
## 1 171.30111 21.11238
## 2  92.84202 62.28205
## 3 148.21905 29.46327
## 4 180.80034 36.56178
## 5  48.35031 93.63930
## 6 217.23071 19.10309
```

```r
### The second file, NOMA-template-A-test.RData, will be used in Question 12.
```

**Question 1.** Plot and interpret the scatterplot of the data and the histograms of $X$ and $Y$. Do you see evidence of heteroskedasticity?

The variance of heating demand can be found visually by observing the vertical dispersion of the observations throughout the year. And we see that the variance depends on the day of the year. In fact, between days 0 and 100, the variance looks stable, but when the heating demand stabilizes for the summer, the variance appears to grow. The low density shown in the histogram of $X$ leads to uncertainty after day 250. The variance looks to decrease after this day, but the interpretation is limited. In conclusion, the data shows evidence of heteroskedasticity.

```r
### An example of how an R-code chunk works.
### The out.width argument gives plots that take up 32% of the line width each.
### The fig.asp = 1 argument gives square plots.

plot(data)
hist(data$x, main = "", freq = FALSE, xlab = "x")
hist(data$y, main = "", freq = FALSE, xlab = "y")
```




**Question 2.** We'd like to estimate the density of $Y$. Plot the Kernel Density Estimator (KDE) with Gaussian kernel and direct plug-in bandwidth. Around which values of $Y$ are we most likely to see bias? And around which values of $Y$ will the KDE be the most variable?

The first objective was to find the optimal bandwidth using the direct plug-in method:

$$\hat{h}_{\text{dpi}} = n^{-1/5}\left[\frac{1}{2\sqrt{\pi}\hat{R}(f'')}\right]^{1/5}$$

The function *bw.SJ* can be used with the argument *method = "dpi"* to find $\hat{h}_{\text{dpi}}$

The next step is to create the plot by evaluating the Gaussian Kernel Density estimator for each point $Y$ on a grid *y_grid*. By connecting the point we formed the curve:

$$\hat{f}(y) = \frac{1}{n * \hat{h}_{\text{dpi}}} \sum_{i=1}^{n} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{y-Y_i}{\hat{h}_{\text{dpi}}}\right)^2}$$

At point 0, a green line is draw to illustrate the boundary issues. Indeed, this is on the left of the line that the bias of our estimation is concentrated. As it's impossible to have a negative heating demand and our estimation gives a non-zero probability estimation to this region. The boundary bias can be solve by transformation, meaning that the data are projected into a $\mathbb{R}$-supported density (via a log or probit transformation). Or it can be solve by boundary kernels. The boundary KDE is plot.

By definition, the bias of the estimation is given by:

$$\text{Bias}\left[\hat{f}(x;h)\right] = \frac{1}{2}\mu_2(K)f''(x)h^2$$

The bias variate in the estimation because of $f''(x)$. The KDE tends to smooth the reality, and it's going to overestimate and underestimate the curvatures. Thus, the estimation has a greater bias in absolute value at the curvature and at the estimations below demand 0.

By definition, the variance of the estimation is given by:

$$\mathrm{Var}\left[\hat{f}(x;h)\right] = \frac{R(K)}{nh} f(x)$$

The variance of our estimation at x depends on the local density of observations. In our case, we can use the *rug* function to see our observations and also the estimation curve. With the *rug* function three key locations with few observations are identified: the lower and upper tails of the KDE and the region around day 70. Therefore the KDE is less variable in these three specific areas.

```
Y = data$y
n = length(Y)
h_dpi = bw.SJ(Y, method = "dpi")
y_grid = seq(-5, 120, length=1000) #min(Y)=8.9875 and max(Y)=108.66

#Kde computation
kdEst_gaussien = function(n, y_grid, Xi, h){
  estimations = sapply(y_grid, function(u) {
    #for each point in the x grid we found the estimation
    z = (u - Xi) / h
    K_val = (1 / sqrt(2 * pi)) * exp(-0.5 * z^2)
    return(sum(K_val) / (n * h))
  })
  return(estimations)
}
y_estimes = kdEst_gaussien(n, y_grid, Y, h_dpi) #Values on the Y axis

# Graph
plot(y_grid, y_estimes, type="l", col="blue", lwd=2,main="KDE Estimation",
     xlab="Total Heating Demand", ylab="Estimated Density")
rug(Y)

########
#Modifcations to see the Boundary issues
abline(v = 0, col = "darkgreen", lty = 2)
idx_neg = which(y_grid < 0)
x_poly = c(min(y_grid), y_grid[idx_neg], 0)
y_poly = c(0, y_estimes[idx_neg], 0)
polygon(x_poly, y_poly, col=rgb(1, 0, 0, 0.3), border=NA)
########

########
# Draw the KDE with the Boundary KDE method to avoid Boundaries issue.
library(evmix)

y_boundary_kde = dbckden(y_grid, Y, lambda = h_dpi, bcmethod = "simple")

#print(y_boundary_kde)
```

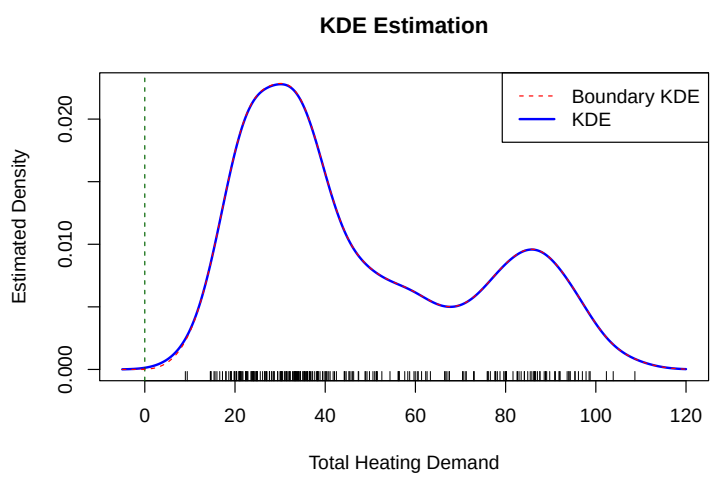
```r
#The first prediction of y_boundary_kde are =0 because the value of
#y_grid are under the demand 0 --> The dbckden() worked

lines(y_grid, y_boundary_kde, type = "l", lty = 2, col = "red")
legend("topright", c("Boundary KDE", "KDE"), lty = c(2, 1), lwd = c(1, 2), col = c("red", "blue
########

########
#Check if density == 1
aire_under_curve<-sum(y_estimes)*(y_grid[2] - y_grid[1])
aire_below_demand0<-sum(y_estimes[y_grid >= 0]) * (y_grid[2] - y_grid[1])
print(aire_under_curve)
```

```
## [1] 0.9999437
```

```r
print(aire_below_demand0) # There is a really small bias boundaries problem
```

```
## [1] 0.9996962
```

```r
########
```




**Question 3.** Compare (and visualize) the KDEs obtained using the normal reference bandwidth (also called Silverman's rule of thumb), the direct plug-in bandwidth, and least-squares cross-validation (also called unbiased cross-validation). How do the different estimators perform close to the boundaries of the support of $Y$?

The comparison of the three KDEs obtained using the three bandwidth selection methods reveals different behaviors. In fact, the Normal Reference rule gives the largest bandwidth ($h_{NR} = 7.59$), resulting in a smoother estimation than the other two methods. In addition, the NR estimator gives a higher density to demand values smaller than 0, meaning that this method suffers from greater

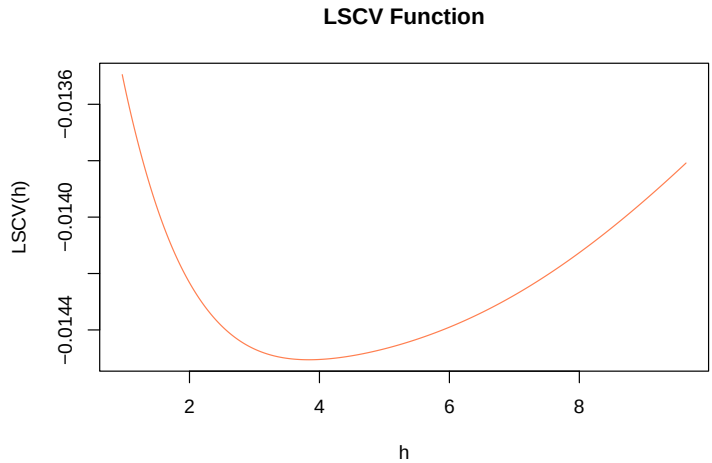
Boundary bias compared to DPI and LSCV. The NR curve is also less responsive to the density peak than the LSCV and DPI curves due to a greater $h_{NR}$.

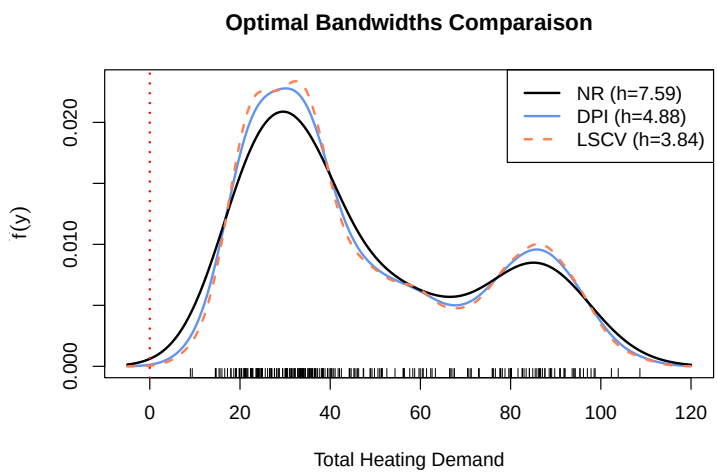
The LSCV and DPI methods both converge towards smaller bandwidths. These estimators are more responsive to the peaks and better track the rapid decay of the density. The $h_{LSCV}$ is the smaller bandwidth and is the more dependent of the small variations of the observations. It explains the double peak between days 20 and 40. The use of a Gaussian kernel implies some probability leakage at the boundaries, in contrast of the NR method, the problem is significantly reduced with the LSCV and DPI bandwidths.

```
Y = data$y
n = length(Y)
y_grid = seq(-5, 120, length=1000)
# We can choose to decrease the length, if it takes too much time.
# -->I prefer to keep it like this to be precise

# KDE gaussien
kdEst_gaussien = function(n, y_grid, Xi, h){
  estimations = sapply(y_grid, function(u) {
    z = (u - Xi) / h
    K_val = (1 / sqrt(2 * pi)) * exp(-0.5 * z^2)
    return(sum(K_val) / (n * h))
  })
  return(estimations)
}

# Silverman's rule (Normal Reference)
h_nr = bw.nrd0(Y)
est_nr = kdEst_gaussien(n, y_grid, Y, h_nr)

# Direct plug-in
h_dpi = bw.SJ(Y, method = "dpi")
est_dpi = kdEst_gaussien(n, y_grid, Y, h_dpi)

# Cross-validation
## Defining the grid
roughness = 0.5/sqrt(pi)
mu2 = 1
h_max = round((243/35 * roughness/mu2^2)^(1/5) * min(sd(Y),
                                                      IQR(Y)/1.349) / n^(0.2), 3)
h_seq = seq(1e-3, h_max, by=1e-3)

## Algorithm
###Constant
K = function(u, kernel="gaussian"){return(dnorm(u))}
Kconv = function(u, kernel = "gaussian"){
  return(dnorm(u, mean = 0, sd = sqrt(2))) # Convolution of two N(0,1) => N(0,2)
}
```

```r
## Leave one out estimation
loo_est = function(h, X, kernel="gaussian"){
  res = sapply(1:n, function(i) K((X[i] - X[-i])/h, kernel))
  return(sum(res)/((n-1)*h))
}

##Square integral of the kernel estimation
intkdEstSq = function(h, X, kernel = "gaussian"){
  res = sapply(X, function(u) Kconv((u-X)/h, kernel))
  return(mean(res)/h)
}

lscv = function(h, X, kernel = "gaussian"){
  intkdEstSq(h, X, kernel) - (2/n) * loo_est(h, X, kernel)
}

## Launch the optimisation
lscv_seq = sapply(h_seq, function(h) lscv(h, Y, kernel = "gaussian"))

# Filter all the really small h
lscv_seq_filtered = lscv_seq[which(h_seq > h_max/10)]
h_seq_filtered = h_seq[which(h_seq > h_max/10)]

## Find the minimum LSCV
h_lscv = h_seq_filtered[which.min(lscv_seq_filtered)]

#Check if it's not a local minimum
plot(h_seq_filtered, lscv_seq_filtered,
     col="coral", type="l", xlab="h", ylab="LSCV(h)", main="LSCV Function")


est_lscv = kdEst_gaussien(n, y_grid, Y, h_lscv)

# Comparative graph
ymax = max(est_nr, est_dpi, est_lscv)

plot(y_grid, est_dpi, type = "l", col = "cornflowerblue",
     lwd = 2, ylim = c(0, ymax),main = "Optimal Bandwidths Comparaison",
     xlab = "Total Heating Demand", ylab = expression(hat(f)(y)))

lines(y_grid, est_nr, col = "black", lwd = 2, lty = 1)
lines(y_grid, est_lscv, col = "coral", lwd = 2, lty = 2)
abline(v = 0, col = "red", lty = 3, lwd=2)
rug(Y)

legend("topright", legend = c(paste("NR (h=", round(h_nr,2), ")", sep=""),
                              paste("DPI (h=", round(h_dpi,2), ")", sep=""),
```

```
                                paste("LSCV (h=", round(h_lscv,2), ")", sep="")),
       col = c("black", "cornflowerblue", "coral"), lty = c(1, 1, 2), lwd = 2)
# The function bw.ucv could also be used to find h_lscv
# bw.ucv(Y) gives approximately the same result
```




**Question 4.** The code below can be used to generate bootstrap 95% confidence intervals for a KDE. Adapt the code to make a plot of the KDE based on the value of $h$ you chose in Question 3 with 95% confidence intervals. Do the intervals become wider or narrower as $h$ increases? Why? Would it be preferable to increase $h$?

We know that the width of the confidence intervals is related to the variance of the estimator. The variance of the Kernel Density Estimator is given by:

$$\mathrm{Var}\left[\hat{f}(x;h)\right] = \frac{R(K)}{nh} f(x)$$

As shown in the formula, the variance is inversely proportional to the bandwidth $h$. Thus, increasing $h$ reduces the variance, and leads to narrower confidence intervals.

However, the choice of bandwidth inserts a certain trade-off between the bias and the variance. In fact, the bias is given by:

$$\text{Bias}\left[\hat{f}(x)\right] = \frac{1}{2}\mu_2(K)f''(x)h^2$$

So, the more $h$ goes up, more biased is the estimation.

In conclusion, if we increase $h$ too much, we obtain a narrow confidence interval, but it will be centered around a biased curve. That is why we prefer a bandwidth that balances these two errors by minimizing the Mean Integrated Squared Error:

$$\text{MISE}[\hat{f}(\cdot;h)] = \frac{1}{4}\mu_2^2(K)R(f'')h^4 + \frac{R(K)}{nh}$$

But the metric depends on the true density $f(x)$ which is unknown. That is why we use the metric LSCV(h), it's an estimator of the MISE:

$$\text{LSCV}(h) = \int \hat{f}(x;h)^2 dx - \frac{2}{n}\sum_{i=1}^{n} \hat{f}_{-i}(X_i;h)$$

```
######
# Choice of the bandwidth method
## with the LSCV metric
K = function(u, kernel="gaussian"){return(dnorm(u))}
Kconv = function(u, kernel = "gaussian"){
  return(dnorm(u, mean = 0, sd = sqrt(2))) # Convolution of two N(0,1) => N(0,2)
}

loo_est = function(h, X, kernel = "gaussian"){
  n = length(X)
  res = sapply(1:n, function(i) K((X[i] - X[-i])/h, kernel))
  return(sum(res)/((n-1)*h))
}


intkdEstSq = function(h, X, kernel = "gaussian"){
  res = sapply(X, function(u) Kconv((u-X)/h, kernel))
  return(mean(res)/h)
}

lscv = function(h, X, kernel = "gaussian"){
  intkdEstSq(h, X, kernel) - (2/n) * loo_est(h, X, kernel)
}
###
Y = data$y
n = length(Y)

h_nr
```

```
## [1] 7.586045
```

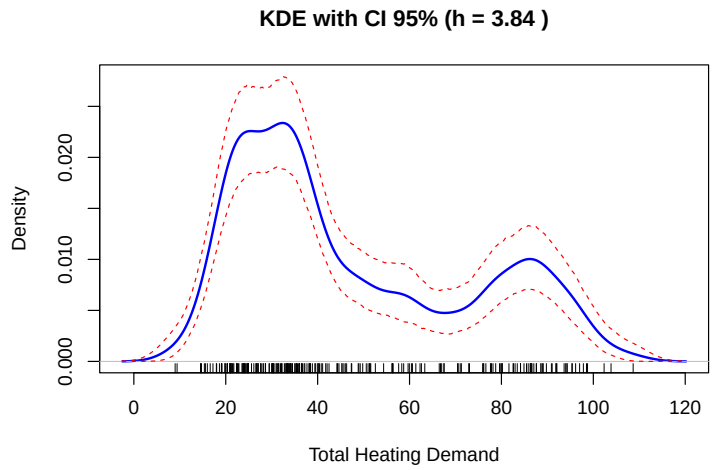
```r
h_dpi
```

```
## [1] 4.884295
```

```r
h_lscv
```

```
## [1] 3.837
```

```r
# Score LSCV
score_nr   = lscv(h_nr, Y)
score_dpi  = lscv(h_dpi, Y)
score_lscv = lscv(h_lscv, Y)
best_h_score = min(score_nr,score_dpi,score_lscv)

#The best is h_lscv --> We will use this one for the example
#####
Y <- data$y
h_chosen <- h_lscv
kde <- density(Y, bw = h_chosen)
boot_y <- function(Y, h, lim){
  new_Y <- sample(Y, replace = TRUE)
  density(new_Y, bw = h, from = lim[1], to = lim[2])$y
}

fits <- replicate(1000, boot_y(Y, h_chosen, range(kde$x)))
CIs <- t(apply(fits, 1, quantile, c(0.025, 0.975)))

# Graph
plot(kde, lwd = 2, col = "blue", ylim = c(0, max(CIs)),
     main = paste("KDE with CI 95% (h =", round(h_chosen,2), ")"),
     xlab = "Total Heating Demand", ylab = "Density")

# Intervals
lines(kde$x, CIs[,1], lty = 2, col = "red")
lines(kde$x, CIs[,2], lty = 2, col = "red")
rug(Y)
```

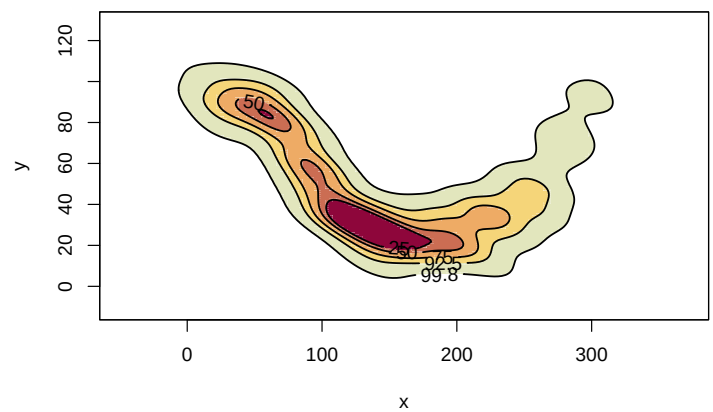



**Question 5.** The code below generates a bivariate gaussian KDE. How can you interpret the plot? This estimator is based on a full bandwidth matrix $H$. Would it be a good idea here to set $H = hI_2$? Why (not)? *NB: you don't need to write any code for this question.*

The plot shows the density estimation using a bivariate KDE. We see in the graph two darker regions. Indeed, this darkness indicates areas with a higher concentration of observations, while the lighter regions at the periphery indicate a low concentration of observations. This is consistent with the histograms from the first question, where we observed a high frequency of data points between day 50 and day 200, this concentration is translated in the graph.

It wouldn't be a good idea to set $H = hI_2$, because this notation implies using the same $h$ for both directions. However, a variation of one unit in time is not comparable to a variation of one unit in heating demand. Thus, this notation is used when the variables are standardized to the same scale. The second argument concerned the correlation, using a scalar matrix restricts the kernels to be circular, forcing the off-diagonal terms to be zero. As we know our data has a complex structure with a negative correlation at the begining, and a postive correlation at the end which might result in a low correlation. But forcing the covariance to be zero is a restriction. That is why we prefer to use a full bandwidth matrix to allow the method to estimate the optimal shape.

```
library(ks)
kde_biv <- kde(data)
plot(kde_biv, display = "filled.contour", cont = c(25, 50, 75, 92.5, 99.75), lwd=1.5)
#print(kde_biv$H)
#--> I wanted to check the correlation, we have here a negative correlation
#because we have more observations in the first part of the year.
```




## 2 Non-parametric regression

The goal is to estimate $m(x) = \mathbb{E}[Y_i \mid X_i = x]$ using different methods and to compare the resulting estimates. You'll may want to use the `KernSmooth` and `splines` packages.

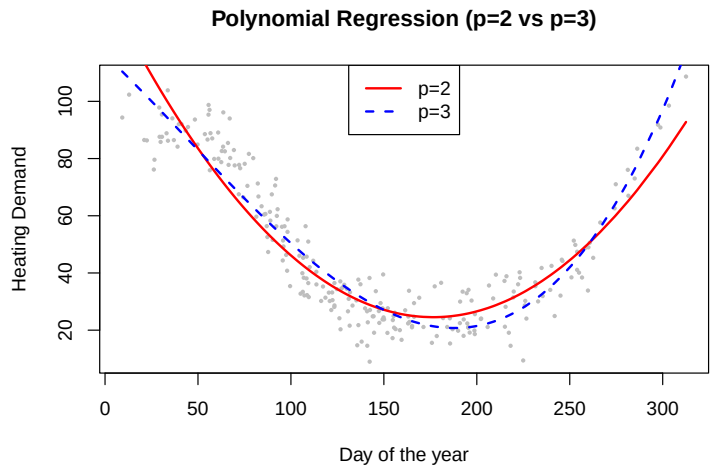
**Question 6.** The scatter plot suggests that a linear model, $m(x) = \beta_0 + \beta_1 x$, is not appropriate. What about a polynomial regression model, $m(x) = \beta_0 + \beta_1 x + \cdots + \beta_p x^p$? Fit this model for $p = 2$ and $p = 3$, plot the resulting estimates and comment on the results. Do you see evidence of heteroskedasticity (and is this in line with your answer to Question 1)?

Polynomial estimators with p=2 and p=3 are more flexible than a linear model, but they remain global estimators. That is why the models try to fit an artificial curvature in regions where the observations looks constant, as bewteen days 125 to 225. In addition, the two estimations are sensitive to boundary effect.In fact, we see at the boundaries that the demand explodes.

The variance of the residuals varies with the day of the year for the two polynomial regressions. This phenomenon indicates heteroskedasticity, the same conclusion as in Question 1.

```
# A Second-degree polynomial regression
fitpol2 <- lm(y ~ x + I(x^2), data = data)

# A third-degree polynomial regression
fitpol3 <- lm(y ~ x + I(x^2) + I(x^3), data = data)

# Graph
plot(data$x, data$y, pch = 16, cex = 0.5, col = "gray",
     main = "Polynomial Regression (p=2 vs p=3)",xlab = "Day of the year",
     ylab="Heating Demand")
order_X_data<- order(data$x) # I use order to have a clear curve
lines(data$x[order_X_data], fitted(fitpol2)[order_X_data],
```

```
      col = "red", lwd = 2)
lines(data$x[order_X_data], fitted(fitpol3)[order_X_data],
      col = "blue", lwd = 2, lty = 2)

legend("top", legend = c("p=2", "p=3"),col = c("red", "blue"),
       lty = c(1, 2), lwd = 2)
```




**Question 7.** Plot the kernel regression (Nadaraya–Watson) estimator for three different values for $h$ (one that looks OK visually, one that is undersmoothing and one that is oversmoothing). Would it be possible to use one of the bandwidths of Question 3 instead? Why (not)?

It would not be a good idea to use the bandwidths from Question 3:

- The bandwidth from question 3 was calculated for density estimation of the response variable $Y$:

  $$\hat{f}(y) = \frac{1}{nh}\sum_{i=1}^{n} K\left(\frac{y - Y_i}{h}\right)$$

  Here, $h$ divides the variable $Y$, so $h$ is expressed in heating demand units.

- For Kernel regression, the estimator is defined as:

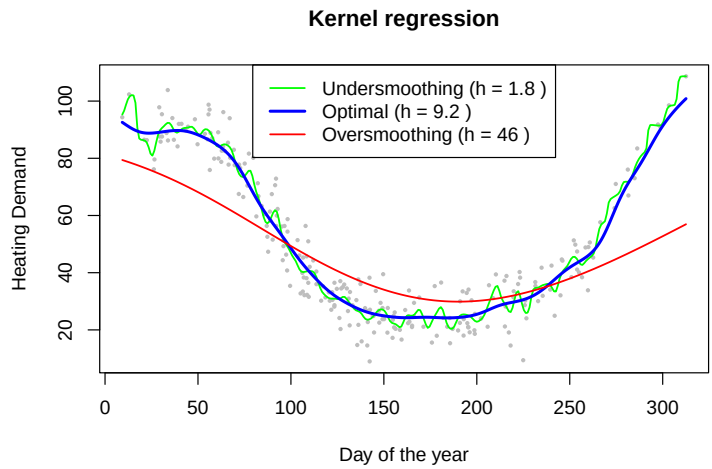
  $$\hat{m}(x) = \frac{\sum_{i=1}^{n} K\left(\frac{x-X_i}{h}\right) Y_i}{\sum_{i=1}^{n} K\left(\frac{x-X_i}{h}\right)}$$

  Here, the kernel acts on the predictor $X$, thus, $h$ is expressed in days.

The two bandwidths also have different objectives, one is trying to minimize the integrated squared error of the density estimator, whereas the other is trying to minimizing the prediction error of the regression function.

In conclusion, the two bandwidths don't have the same objective and the same scale and it would not be good idea.

```r
library(KernSmooth)

K = function(u, kernel = "gaussian"){
  return(dnorm(u))
}

nwEst = function(x, X, Y, h, kernel = "gaussian"){
  weights = sapply(x, function(u) K((u-X)/h, kernel = kernel))
  res = crossprod(weights, Y)
  sumWeights = colSums(weights)
  sumWeights[sumWeights == 0] = 1
  return(res/sumWeights)
}
h_optimal_val <- dpill(data$x, data$y)

h_ok    <- h_optimal_val
h_under <- h_ok / 5
h_over  <- h_ok * 5


x_grid <- seq(min(data$x), max(data$x), length.out = 300)

est_ok    <- nwEst(x_grid, data$x, data$y, h_ok)
est_under <- nwEst(x_grid, data$x, data$y, h_under)
est_over  <- nwEst(x_grid, data$x, data$y, h_over)

plot(data$x, data$y, pch = 16, cex = 0.5, col = "gray",
     main = "Kernel regression",xlab = "Day of the year",
     ylab = "Heating Demand")

lines(x_grid, est_under, col = "green", lwd = 1.5)
lines(x_grid, est_ok,    col = "blue",  lwd = 2.5)
lines(x_grid, est_over,  col = "red",   lwd = 1.5)

legend("top", legend = c(paste("Undersmoothing (h =", round(h_under,1), ")"),
                         paste("Optimal (h =", round(h_ok,1), ")"),
                         paste("Oversmoothing (h =", round(h_over,1), ")")),
       col = c("green", "blue", "red"), lwd = c(1.5, 2.5, 1.5))
```




**Question 8.** Fit a local linear estimator (i.e. a local polynomial estimator with $p = 1$) with a bandwidth $h$ chosen as the direct plug-in bandwidth $\hat{h}_{\mathrm{rt}}$. In which regions do we expect the estimator to be the least biased? And where do we expect it to have the smallest variance? Compare the result with the Nadaraya–Watson estimate of the previous question (use the same $h$) and comment on the differences.

The graph shows that the Nadaraya-Watson estimator and the local linear estimator behave similarly. However, differences appear at the boundaries (before the day 50, and after day 250). It could be explained by examining the asymptotic bias for both estimators:

$$B_p(x) = \begin{cases} \frac{\mu_2(K)}{2}\left\{m''(x) + \frac{2m'(x)f'(x)}{f(x)}\right\} & \text{if } p = 0, \\ \frac{\mu_2(K)}{2} m''(x) & \text{if } p = 1. \end{cases}$$

The Nadaraya-Watson estimator includes an additional term in the bias formula ($\frac{2m'(x)f'(x)}{f(x)}$). This term becomes problematic at the boundaries where the density of observations is asymmetric. In fact, at the beginning, there are more observations on the right part of the graph than on the left and, at the end, there are more observations on the left than on the right. At the beginning of the graph we have more observations on the right and the demand is decreasing, that is why the estimation is dragged down. The same principle appears at the end.

The term $m''(x)$ is common to both estimators. The term also depends on x, indeed, when the function is nearly linear, $m''(x) \approx 0$, but in regions with high curvature the term becomes large. In the graph, between days 150 and 200, the curvature is high, it's translated a high $|m''(x)|$ and a high bias in absolute value.

For both estimators, the variance is given by:

$$\mathrm{Var}[\hat{m}(x; p, h)|X_1, \ldots, X_n] \approx \frac{R(K)}{nhf(x)}\sigma^2(x)$$

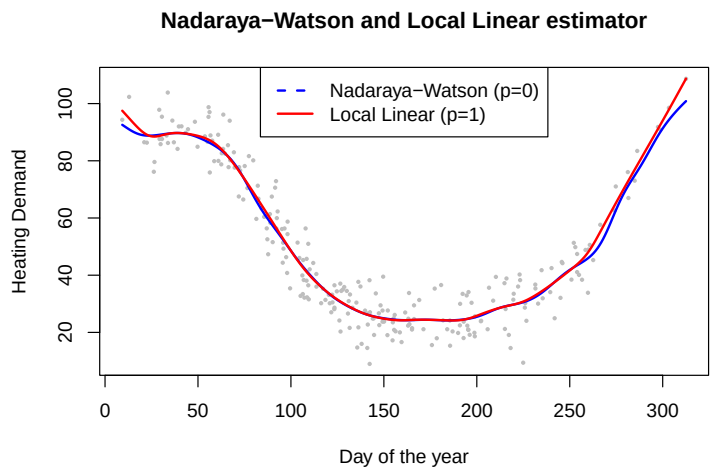
The denominator $\frac{1}{nhf(x)}$ represents the effective number of local observations. If the density ($f(x)$) is high, it means that we look at more data than it lowers the variance down. A larger $h$ reduces

the variance because we are looking at a wider window containing more points. The roughness of the kernel is constant and depends on the kernel choice. The $\sigma^2(x)$ represents the intrinsic noise of the response variance $Y$.

Based on the formula, we can now identify the variance across the graph. We observe a high variance at the beginning of the year, in fact, the $\sigma^2(x)$ is high and the density $f(x)$ is relatively low. Between days 50 to 100, the variance decreases , as the $\sigma^2(x)$ drops and the density of observations is increasing. Between days 100 to 225, we observe a complex variance as the noise is extremely high and the density is high. And the variance of the end of the graph is also complex as the noise decreases with the density.

```r
library(KernSmooth)

h_rt <- dpill(data$x, data$y)
x_grid <- seq(min(data$x), max(data$x), length.out = 300)


#local linear
lpReg = function(x, X, Y, h, p, kernel = "gaussian"){
  D = sapply(x, function(u) outer(X-u, 0:p, `^`), simplify = "array")
  W = sapply(x, function(u) K((u - X)/h, kernel = kernel)/h)
  b = sapply(1:length(x),
             function(i)
               solve(t(D[,,i])%*%diag(W[,i])%*%D[,,i])%*%t(D[,,i])%*%diag(W[,i])%*%Y)
  if (p >= 1){
    return(b[1,]) # \hat{\beta}_0(x) = \hat{m}(x)
  } else return(b)
}
est_ll <- lpReg(x_grid, data$x, data$y, h_rt, p = 1) #local linear

#Nadaraya-Watson
nwEst = function(x, X, Y, h, kernel = "gaussian"){
  weights = sapply(x, function(u) K((u-X)/h, kernel = kernel))
  res = crossprod(weights, Y)
  sumWeights = colSums(weights)
  return(res/sumWeights)
}
est_nw <- nwEst(x_grid, data$x, data$y, h_rt, kernel = "gaussian") #Nadaraya-Watson

plot(data$x, data$y, pch = 16, cex = 0.5, col = "gray",
     main = "Nadaraya-Watson and Local Linear estimator",
     xlab = "Day of the year", ylab = "Heating Demand")

lines(x_grid, est_nw, col = "blue", lwd = 2)#Nadaraya-Watson
lines(x_grid, est_ll, col = "red",  lwd = 2)#local linear

legend("top", legend = c(paste("Nadaraya-Watson (p=0)"),
```

```
                        paste("Local Linear (p=1)")),
       col = c("blue", "red"), lty = c(2, 1), lwd = 2)
```




**Question 9.** Fit and visualize the $k$NN regression estimator with tricube kernel and $p = 2$ for three different values of $k$ (again, try to pick one that is visually convincing, one that's undersmoothing and one that's oversmoothing). Does it look like a good competitor of the local linear estimator above?

The $k$NN estimator is a very strong competitor to the local linear estimator. In question 8, we observed that the Nadaraya–Watson estimator has a high bias at the boundaries and at the peaks. While the local linear estimator corrects the bias at boundaries, but it still suffers from a bias with peaks.

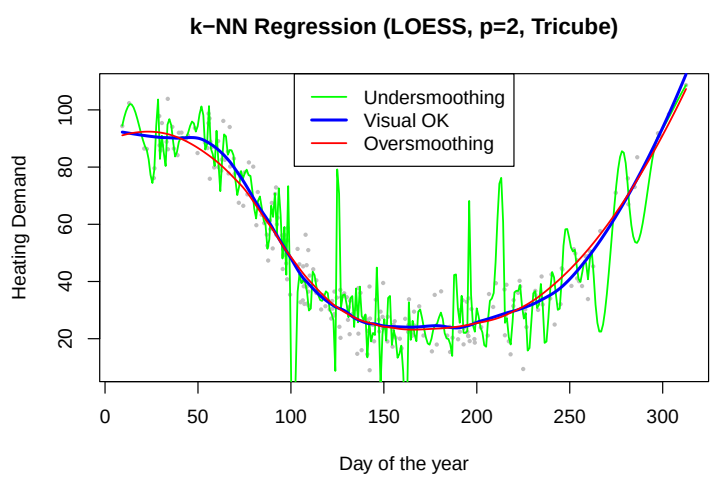
The $k$NN regression estimator with tricube kernel and $p = 2$ has two advantages. First, it models the curvature as $p = 2$ and reduce the bias in the region. The estimator also uses a fixed number of neighbors $k$, this implies that the bandwidth adapts locally :

$$h(x) \propto \frac{k}{n f_X(x)}$$

Consequently, regions with low data density are less problematic. In fact, when the region has low data, the bandwidth automatically grows to include enough neighbors. However, $k$NN reduces boundary and curvature bias, but the regions with very low data may still suffer from high variance. In conclusion, it's a strong competitor but its performance depends on the local structure of the observations.

```
# Choice of k
k_under <-5
k_ok <-55
k_over <-150
```

```r
#Span
span_under <- k_under/n
span_ok    <- k_ok/n
span_over  <- k_over/n

# Fit with loess (tricube kernel by default)
## p=2
fit_under <- loess(y ~ x, data = data, degree = 2, span = span_under)
fit_ok    <- loess(y ~ x, data = data, degree = 2, span = span_ok)
fit_over  <- loess(y ~ x, data = data, degree = 2, span = span_over)

x_grid <- seq(min(data$x), max(data$x), length.out = 300)

pred_under <- predict(fit_under, x_grid)
pred_ok    <- predict(fit_ok, x_grid)
pred_over  <- predict(fit_over,x_grid)

# Graph
plot(data$x, data$y, pch = 16, cex = 0.5, col = "gray",
     main = "k-NN Regression (LOESS, p=2, Tricube)",xlab = "Day of the year",
     ylab = "Heating Demand")

lines(x_grid, pred_under, col = "green", lwd = 1.5)
lines(x_grid, pred_ok,    col = "blue",  lwd = 2.5)
lines(x_grid, pred_over,  col = "red",   lwd = 1.5)

legend("top", legend = c(paste("Undersmoothing"), paste("Visual OK"),
                         paste("Oversmoothing")),col = c("green", "blue", "red")
       , lwd = c(1.5, 2.5, 1.5))
```




**Question 10.** Use cross-validation to pick an optimal bandwidth for the local linear estimator and

an optimal number of neighbours $k$ for the $k$NN estimator. For the local linear estimator, you may use the code provided in TP2 or the function `npregbw` in the package `np` (see example below). For the $k$NN estimator, you may use the `cvNN` function below.

Make a plot showing simultaneously (i) the polynomial regression estimator of Question 6 (ii) the local linear estimator with CV bandwidth and (iii) the $k$NN estimator with CV bandwidth. Can you decide which one of the three estimators you'd prefer based on the plot? Do you think the CV criterion selected appropriate values for $h$ and $k$?

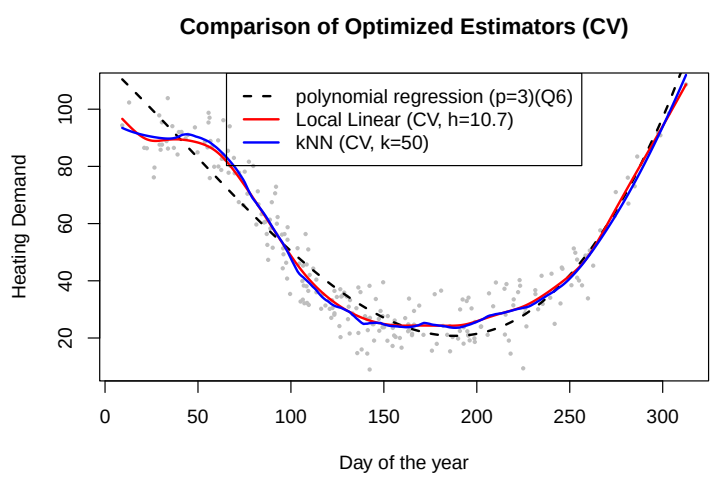
The global polynomial from question 6 is likely the least preferred. In fact, the model is too rigid to capture local variations. The rigidity of the estimation could be seen on the graph at the peak.

The local linear and $k$NN estimators perform better than the global polynomial form, and have a similar estimation. But as discussed in question 9, the $k$NN better capture the varying density of observations slightly better, as it could be seen at the beginning of the graph. But, visually, despite the first part, they follow a similar trend.

The bandwidth and the $k$ selected by the CV criterion minimizes the estimated Mean Squared Error, and balance the bias-variance trade off. The estimations appear to well estimate the observations. However, since the observations have heteroskedasticity, the CV criterion may not be optimal. That is why the optimal $h$ and $k$ found by the CV criterion should be interpreted with caution.

```r
library(np)
options(np.messages = FALSE)
# npregbw(y ~ x, data = data, regtype = "ll")

x_grid <- seq(min(data$x), max(data$x), length.out = 300)

# Optimal k
k_grid <- seq(1, 50, by = 1)

cvNN <- function(X, Y, k){
  n <- length(X)
  CVknn <- numeric(length = n)
  for (i in 1:n) {
    train_data <- data.frame(X = X[-i], Y = Y[-i])
    Ypred <- predict(loess(Y ~ X, data = train_data, span = k/(n-1),
            control = loess.control(surface = "direct")), X[i])
    CVknn[i] <- (Y[i] - Ypred)^2
   }
  return(mean(CVknn))
}

cv_errors_k <- sapply(k_grid, function(k) cvNN(data$x, data$y, k))
k_opt <- k_grid[which.min(cv_errors_k)] # Number of k that minimises the error

fit_knn <- loess(y ~ x, data = data, span = k_opt/n, degree = 2,
                 control = loess.control(surface = "direct"))
pred_knn <- predict(fit_knn, data.frame(x = x_grid))
```

```r
# Optmal h
bw_object <- npregbw(y ~ x, data = data, regtype = "ll")
h_opt <- bw_object$bw

fit_ll <- npreg(bw_object,x_grid,data.frame(x = x_grid))
pred_ll <- fitted(fit_ll)

# A third-degree polynomial regression (from Q6)
fitpol3 <- lm(y ~ x + I(x^2) + I(x^3), data = data)
order_X_data<- order(data$x) # I use order to have a clear curve

# Graph
plot(data$x, data$y, pch = 16, cex = 0.5, col = "gray",
     main = "Comparison of Optimized Estimators (CV)",xlab = "Day of the year",
     ylab = "Heating Demand")

# (i)
lines(data$x[order_X_data],
      fitted(fitpol3)[order_X_data], col = "black", lwd = 2, lty = 2)

# (ii)
lines(x_grid, pred_ll, col = "red", lwd = 2)

# (iii)
lines(x_grid, pred_knn, col = "blue", lwd = 2, lty = 1)

legend("top", legend = c("polynomial regression (p=3)(Q6)",
                         paste("Local Linear (CV, h=", round(h_opt,1), ")", sep=""),
                         paste("kNN (CV, k=", k_opt, ")", sep="")),
       col = c("black", "red", "blue"), lwd = 2, lty = c(2, 1, 1))
```



**Question 11.** Fit a natural cubic spline estimator (you can use the `ns` function from the `splines` package) for (i) 4 equally-spaced interior knots and (ii) 4 knots chosen as equidistance quantiles of the covariate. Please comment on the fit. To how many degrees of freedom do these models correspond?

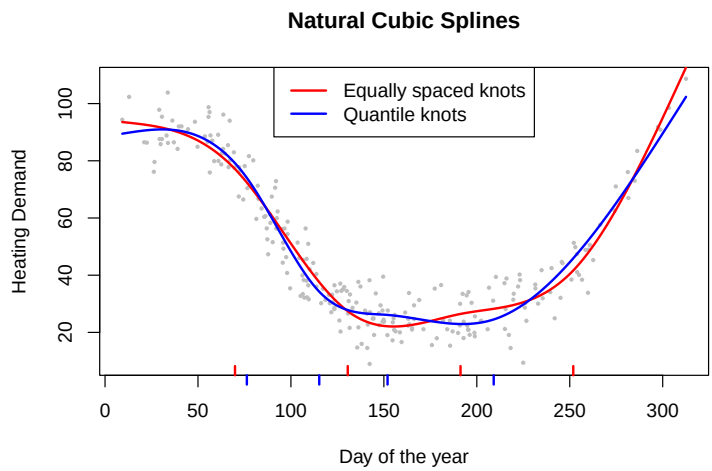
The cubic spline estimator with 4 equally-spaced interior knots is a naive approach. In fact, the knots are placed blindly. This risks placing knots in low data regions with not enough information, and this leads to high variance. There also might not be enough knots in the center of the graph to capture all the available information, and induces bias.

The cubic spline estimator with 4 knots chosen as equidistant quantiles is more appropriate The knots are concentrated in the highest density regions of the graph, this allows more flexibility and reduce the bias.

A natural cubic splines estimator with $K$ knots have $K + 2$ degrees of freedom. Thus, the degrees of freedom for these models is equal to 6.

```r
library(splines)

x_grid <- seq(min(data$x), max(data$x), length.out = 300)

# Creating the 4 Knots and 5 range (equidistance)
range_x <- range(data$x)
knots_eq <- seq(range_x[1], range_x[2], length.out = 6)[2:5]

# Creating the 4 Knots and 5 range (statistic repartition)
knots_qu <- quantile(data$x, probs = c(0.2, 0.4, 0.6, 0.8))

# fits natural splines
fit_ns_eq <- lm(y ~ ns(x, knots=knots_eq), data = data)
fit_ns_qu <- lm(y ~ ns(x, knots=knots_qu), data = data)


# Prediction
pred_eq <- predict(fit_ns_eq, data.frame(x = x_grid))
pred_qu <- predict(fit_ns_qu, data.frame(x = x_grid))

# Graph
plot(data$x, data$y, pch = 16, cex = 0.5, col = "gray",
     main = "Natural Cubic Splines",xlab = "Day of the year",
     ylab = "Heating Demand")

lines(x_grid, pred_eq, col = "red", lwd = 2)
lines(x_grid, pred_qu, col = "blue", lwd = 2)

# Add the positions of the knots
rug(knots_eq, col = "red", lwd = 2, ticksize = 0.03)
rug(knots_qu, col = "blue", lwd = 2, ticksize = -0.03) # Vers le bas
```

```r
legend("top",legend = c("Equally spaced knots", "Quantile knots"),
       col = c("red", "blue"),lwd = 2,lty = 1)
```




**Question 12.** Fit a smoothing spline estimator with $\lambda$ chosen by cross-validation (using the function below). Plot your grid of values of $\lambda$ against CV($\lambda$). What is the effective degrees of freedom of the model with this choice of $\lambda$? Doe the fit look better than that of the natural cubic spline?

The effective degrees of freedom decreases as the smoothing parameter $\lambda$ increases. The value of $\lambda$ obtained by cross-validation is equal to 0.001 with a corresponding effective degrees of freedom of 8.65.

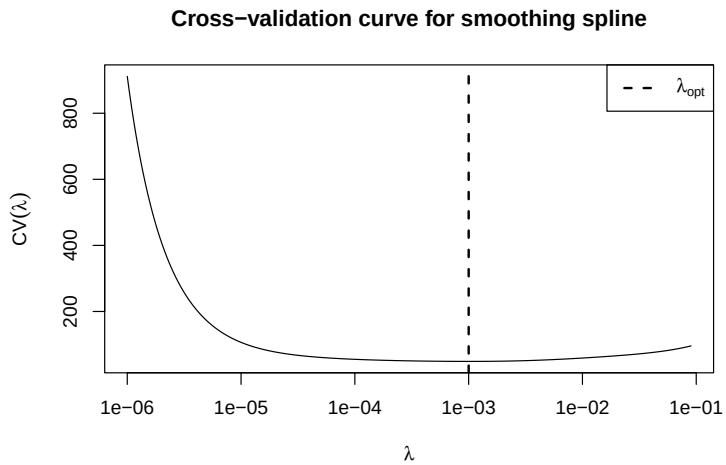
The degrees of freedom is fixed by the number of knots in the natural cubic spline, and in the smoothing spline the degrees of freedom is optimized through $\lambda$. Thus, the smoothing spline provides a greater control on the balance between bias and variance than the natural cubic spline.

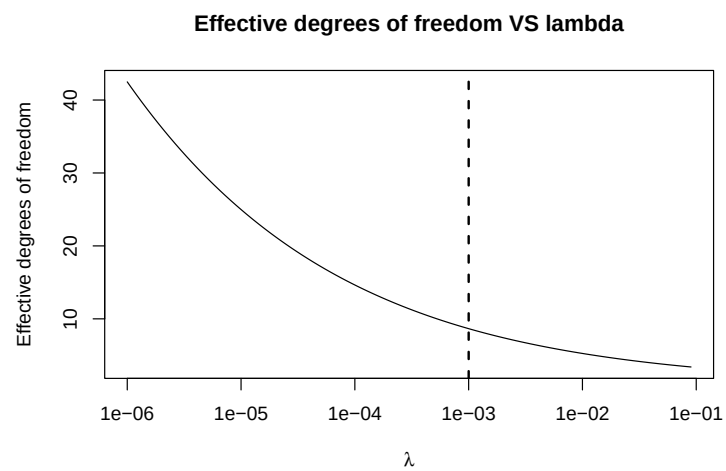
```r
lambda_grid <- exp(seq(log(1e-6), log(0.09) , length.out = 1000)) # Log grid
cv_values <- numeric(length(lambda_grid))
# Vector to create the graph of CV
df_values <- numeric(length(lambda_grid))
# Vector to create the graph of degrees of freedom

# Croos validation Lambda
cv_lambda <- function(x, y, lambda) {
  fit <- smooth.spline(x, y, lambda = lambda, all.knots = TRUE)
  yhat <- predict(fit, x)$y
  h <- fit$lev
  res <- (y - yhat) / (1 - h)
  return(mean(res^2))
}
```

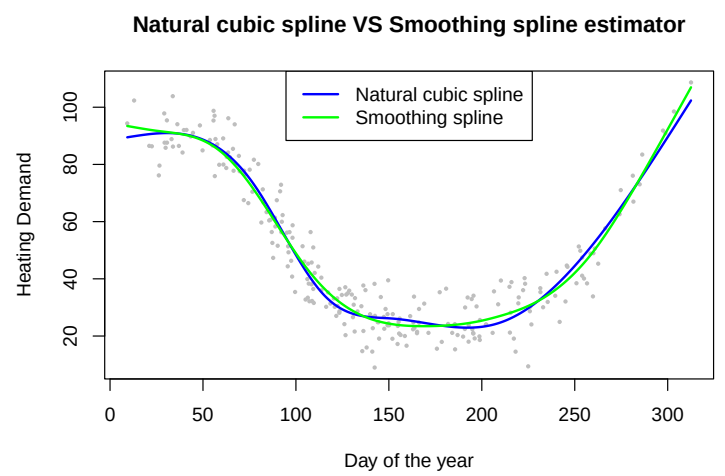
```r
# Looping on Lambda grid
for (i in 1:length(lambda_grid)) {
  fit <- smooth.spline(data$x, data$y, lambda = lambda_grid[i], all.knots = TRUE)
  yhat <- predict(fit, data$x)$y
  h <- fit$lev
  cv_values[i] <- mean(((data$y - yhat) / (1 - h))^2)
  #Store the CV score for each lambda
  df_values[i] <- fit$df  #Store the df values for each lambda
}

lambda_opt <- lambda_grid[which.min(cv_values)] #Better lambda
fit_smooth <- smooth.spline(data$x, data$y,lambda = lambda_opt,all.knots = TRUE)
fit_smooth$df #Degree of freedom
```

```
## [1] 8.652382
```

```r
pred_smooth <- predict(fit_smooth, x_grid)$y

# Graph CV
plot(lambda_grid, cv_values, type = "l",log = "x",
     xlab = expression(lambda),ylab = expression(CV(lambda)),
     main = "Cross-validation curve for smoothing spline")
abline(v = lambda_opt, col = "black", lwd = 2, lty = 2)
legend("topright",legend = expression(lambda[opt]),col = "black",lwd = 2,lty = 2)

# Graph degrees of freedom
plot(lambda_grid, df_values, type = "l",log = "x",xlab = expression(lambda),
     ylab = "Effective degrees of freedom",
     main = "Effective degrees of freedom VS lambda")
abline(v = lambda_opt, col = "black", lwd = 2, lty = 2)

# Graph on the observations
plot(data$x, data$y, pch = 16, cex = 0.5, col = "gray",
     main = "Natural cubic spline VS Smoothing spline estimator",
     xlab = "Day of the year", ylab = "Heating Demand")
lines(x_grid, pred_qu, col = "blue", lwd = 2)
lines(x_grid, pred_smooth, col = "green", lwd = 2)
legend("top",legend = c("Natural cubic spline", "Smoothing spline"),
       col = c("blue", "green"),lwd = 2)
```




**Question 13.** Focus on at least **three** of the previously fitted models, i.e., pick at least three models among the

- polynomial regression of degree $p$ (Question 6);
- local linear estimator with cross-validation or direct plug-in bandwidth (Question 8 and 10);
- $k$NN regression estimator with $k$ chosen via cross-validation (Questions 9 and 10)
- natural cubic spline estimator with your choice and number of knots (Question 11)
- smoothing spline with $\lambda$ chosen via cross-validation (Question 12).

Compare these models in terms of their **predictive performance** as follows. You've estimated the above models based on a dataset of size `n = 250`, the *training data*. The file `NOMA-template-A-test.RData` contains the *testing data*, i.e., `N = 50` additional observations that were not part of the training data. For each fitted regression model, you can now calculate predictions $\hat{y}_i$, associated with covariate observations $x_i$ from the testing data, and calculate the **Mean Squared Prediction Error** (MSPE),

$$\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Report the MSPE for the models you considered and conclude which model performs best according to this criterion. Is this in line with your previous conclusions?

In the comparison of predictive performance between three different models, I selected: the global polynomial regression (p=3)(from Q6), the local linear estimator with direct plug-in bandwidth (from Q8), and the smoothing spline with $\lambda$ chosen via cross-validation (Q12).

The global polynomial regression was not very performant as it has a result of $MSPE = 74.26$. Global parametric models are rigid and have problems capturing complex variation. In fact, the model cannot capture the local variations of heating demand.

The local linear estimator achieves the best score ($MSPE = 49.73$), and slightly outperforming the smoothing spline model, which received a score of $MSPE = 51.51$. The smoothing spline method is a sophisticated and non-parametric estimation. Its parameters were optimized via Cross-validation, which selects the values that best balance the bias-variance trade-off. But the local linear estimator better approximates the new observations from the test set.

In conclusion, the local linear estimator is the preferred model among these three based on the MSPE criterion.

```r
### Load data_test here using:
name <- "40852100-template-A-test.RData"
load(name)
head(data_test)
```

```
##            x        y
## 1 120.121207 32.11511
## 2   9.127896 95.72100
## 3  43.390322 84.56581
## 4  96.216242 47.95087
## 5  57.563320 90.06040
## 6  83.291307 60.63373
```

```r
### If you want to "predict" from a locpoly fit, you'll need to interpolate:
# fit <- locpoly(data$x, data$y, bandwidth = 15)
# f_pred <- approxfun(fit$x, fit$y)
# y_hat <- f_pred(data_test$x)

mspe <- function(y_true, y_pred) {
  mean((y_true - y_pred)^2)
}

# Polynomial regression (Q6)
pred_pol3 <- predict(fitpol3, newdata = data.frame(x = data_test$x))
mspe_pol3 <- mean((data_test$y - pred_pol3)^2)
mspe_pol3 #=74.26
```

```
## [1] 74.26152
```

```r
# Local linear(Q8)
h_rt <- dpill(data$x, data$y)
pred_ll <- lpReg(data_test$x, data$x, data$y, h_rt, p = 1)
mspe_ll <- mean((data_test$y - pred_ll)^2)
mspe_ll #=49.73
```

```
## [1] 49.7318
```

```r
# Smoothing spline (Q11)
fit_smooth <- smooth.spline(data$x, data$y,lambda = lambda_opt,all.knots = TRUE)
pred_smooth <- predict(fit_smooth, data_test$x)$y
mspe_smooth <- mean((data_test$y - pred_smooth)^2)
mspe_smooth #51.51
```

```
## [1] 51.50924
```